



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



Vizsgaremek

Webalapú játékkönyvtár fejlesztése egy- és helyi többjátékos
támogatással

Development of a web-based game library with single- and local
multiplayer support

Árvai Márk Szabolcs
Galó Gellért



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



Vizsgaremek adatlap

A Vizsgaremek készítői:

Név: Árvai Márk Szabolcs
E-mail címe: mark.szabolcs.arvai@umszki.hu

Név: Galó Gellért
E-mail címe: galo.gellert@umszki.hu

A Vizsgaremek témája:

Egy modern, webalapú játékkönyvtár létrehozása, melyben a felhasználók, regisztrációt követően egyedül, illetve többedmagukkal egyaránt tudnak bizonyos játékokat játszani.

A Vizsgaremek címe:

Webalapú játékkönyvtár fejlesztése egy- és többjátékos támogatással

Development of a web-based game library with single- and multiplayer support

Konzulens:

Deák László

Dátum:

.....
aláírás

.....
aláírás

.....
aláírás



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



Eredetiségi nyilatkozat

Alulírott tanuló kijelentem, hogy a vizsgaremek saját és csapattársa(i)m munkájának eredménye, a felhasznált szakirodalmat és eszközöket azonosíthatóan közöltem. Az elkészült vizsgaremekrészt képező anyagokat az intézmény archiválhatja és felhasználhatja.

Dátum:

.....
Árvai Márk Szabolcs
Tanuló aláírása

.....
Galó Gellért
Tanuló aláírása



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



Konzultációt igazoló lap

Alulírott Deák László konzulens aláírással igazolom Árvai Márk és Galó Gellért nevű tanulók konzultációkon való részvételét.

Dátum	Téma	Tanuló aláírása	Konzulens aláírása
2025.10.28. 16:30-17:45	Projekt iránymutatás, célok leszűkítése Kriston Dávid tanárral		
2025.11.17. 16:00-17:00	Vizsgaremek dokumentációjának megbeszélése, projekt állapotának bemutatása		
2025.12.08. 16:00-17:00	Projekt állapotának bemutatása		
2025.12.15. 16:00-17:00	Projekt állapotának bemutatása		
2026.01.05. 16:00-17:00	Projekt végleges formájának bemutatása		
2026.01.26. 17:30-17:45	Prezentációs konzultáció		



Tartalomjegyzék

Vizsgaremek adatlap.....	2
A Vizsgaremek készítői:	2
A Vizsgaremek témája:	2
A Vizsgaremek címe:	2
Konzulens:.....	2
Eredetiségi nyilatkozat.....	3
Konzultációt igazoló lap	4
1. Projekt ismertetése	6
2. Megrendelő bemutatása	10
3. Követelmények és célkitűzések	13
4. Rendszerelemzés és tervezés	16
5. Feladatok meghatározása és erőforrás-tervezés.....	22
6. Fejlesztői környezet és eszközök	27
7. Megvalósítás	30
8. Felhasználói dokumentáció.....	43
8. Fejlesztői dokumentáció	47
9. Továbbfejlesztési lehetőségek	59
10. Összegzés	60
11. Források	61



1. Projekt ismertetése

Egy innovatív, webalapú játékkönyvtár fejlesztése

A fejlesztett rendszer egy webalapú játékkönyvtár, amely lehetővé teszi, hogy a felhasználók regisztráció után egy biztonságos, adatbázissal támogatott környezetben különböző digitális játékokhoz férjenek hozzá. A platform egy központi felületet biztosít, ahol a klasszikus és modern játékok egyaránt elérhetők, egyjátékos és helyi többjátékos módban.

A főoldalon egy vizuális játékkatalógus jelenik meg, amelyben a játékok kártyák formájában érhetők el. A felhasználók keresőmező segítségével gyorsan megtalálhatják a kívánt játékot, majd egy kattintással elindíthatják azt. Egyes játékok közvetlenül a böngészőben futnak – például a TRON –, míg mások letölthető formában (.exe) érhetők el külső forrásból, például GitHubon keresztül.

A felhasználók saját profillal rendelkeznek, amely megjeleníti az elért eredményeket, statisztikákat és a legutóbbi játékokat. A profil személyre szabható, például profilkép (avatar) és profilhátter kiválasztásával, így a rendszer egyéni megjelenést és azonosíthatóságot biztosít minden játékos számára. A pontszámok és eredmények játék függően ranglistákon (leaderboard) is megjelennek, ami különösen a Space Shooter típusú játékoknál fontos.

A platform tartalmaz szabály- és ismertetőoldalakat is, ahol a felhasználók megismerhetik az egyes játékok működését és szabályrendszerét. Ezek az oldalak segítik az új játékosokat a gyors bekapcsolódásban, és biztosítják, hogy minden játék egységesen dokumentált legyen.

A TRON játék egy böngészőben futó, kétjátékos arénajáték, amely egy közös pályán zajlik, a klasszikus „lightbike” játékmenet alapján. A játékosok ugyanazon a képernyőn, ugyan abban az arénában mérettethetik meg magukat egymás ellen, miközben a rendszer a mozgást, az ütközéseket és az eredményeket valós időben kezeli.

A rendszer működésének alapját egy webes felület és egy adatbázis-alapú háttérrendszer adja. Az adatbázis tárolja a felhasználói fiókokat, a játékokhoz tartozó eredményeket és statisztikákat, míg a webes felület biztosítja a játékok elérését, a profilkezelést és a ranglisták megjelenítését. A platform moduláris felépítése lehetővé teszi új játékok és funkciók későbbi hozzáadását, így a rendszer hosszú távon is bővíthető marad.



Project Description

Development of an Innovative Web-Based Game Library

The developed system is a web-based game library that allows users, after registration, to access a variety of digital games in a secure, database-supported environment. The platform provides a central interface where both classic and modern games are available in single-player and local multiplayer modes.

On the main page, users are presented with a visual game catalog where each game appears as a selectable tile. A built-in search function allows players to quickly find a specific title and launch it with a single click. Some games run directly in the browser, such as the TRON game, while others are available as downloadable applications (.exe) hosted on external platforms such as GitHub.

Each user has a personal profile that displays their game history, scores, and statistics. Profiles can be customized with avatars and background images, allowing players to personalize their presence on the platform. For supported games, such as Space Shooter, scores are also displayed on public leaderboards, enabling competitive comparison between users.

The platform includes rule and information pages for each game, where players can read how the games work and learn their rules before playing. These pages ensure that every game is clearly documented and accessible even to new users.

The TRON game is a browser-based two-player arena game inspired by the classic light-cycle arcade style. Both players compete on a shared arena, while the system handles movement, collision detection, and scoring in real time.

The system is built on a web interface combined with a database-driven backend. The database stores user accounts, game results, and statistics, while the web interface provides access to games, profile management, and leaderboards. Thanks to its modular design, the platform is prepared for future expansion with new games and additional features, allowing it to remain flexible and scalable over time.



A projekt célja és a téma választásának indoklása

A projekt célja egy olyan weboldal létrehozása, amely a játékok modern, online környezetben történő újraértelmezését valósítja meg. A fejlesztés központi eleme, hogy a felhasználók regisztrációt követően biztonságos, adatbázis által kezelt felületen játszhassanak egymás ellen, vagy egyedül. A rendszer célja, hogy a hagyományos játékelményt digitális formában, elérhető és közösségi módon tegye újraélhetővé.

A projekt különlegessége, hogy nem csupán online játékelményként működik, hanem egyben digitális archívumként is, amely megőrzi és újraéleszti a klasszikus játékok formavilágát és játékmenetét. A platform lehetőséget ad arra, hogy a régi játékok ne merüljenek feledésbe, hanem modern, technológiailag korszerű módon tovább éljenek. Ezzel a fejlesztés nemcsak szórakoztatási funkciót lát el, hanem hozzájárul a digitális kulturális örökség megőrzéséhez is.

Téma választásának indoklása, motiváció

A téma választása mögött személyes és szakmai motiváció is áll. A videojátékok, valamint a hagyományos logikai játékok iránti érdeklődés és a digitális archiválás iránti elkötelezettség inspirálta a fejlesztést. A cél egy olyan rendszer megalkotása volt, amely technikailag is kihívást jelent, de közben értéket is teremt: egyszerre szolgálja a közösségi játékelményt, a tanulást és a múlt megőrzését.

A projekt hasznossága

A fejlesztett weboldal több szinten is hasznosítható, mivel egyszerre szolgál szórakoztatási, oktatási és kulturális célokat. A platform elsődleges célcsoportja a videojátékok és logikai játékok iránt érdeklődő felhasználók köre, akik modern online környezetben szeretnék újraélni a klasszikus játékok hangulatát. A rendszer böngészőből elérhető, így bárki számára hozzáférhető, legyen szó otthoni játékosról, diákokról vagy akár idősebb felhasználókról, akik nosztalgikus élményeket keresnek.

Az oldal hasznos eszköz lehet oktatási intézmények számára is, mivel a játékokon keresztül fejleszthetők a logikai, stratégiai és problémamegoldó készségek. A felület bevezethető iskolai informatika- vagy logikai játékokkal kapcsolatos tanórákon, illetve szakkörökben is.

Ezen túlmenően a platform archiválási funkciója révén értékes forrás lehet kutatóknak, gyűjtőknek és játéktörténettel foglalkozó szakembereknek. A régi játékok megőrzésével és



dokumentálásával a projekt hozzájárul a digitális kulturális örökség hosszú távú megőrzéséhez, valamint elősegíti a magyar játékipar múltjának és fejlődésének feldolgozását.

A fejlesztés tehát nemcsak egy szórakoztató webalkalmazás, hanem egy olyan komplex rendszer, amely több célcsoport számára is értéket teremt — játékosoknak, oktatóknak és archiválási szakembereknek egyaránt.

A fejlesztett rendszer funkciói

Játékfunkciók:

- Egyjátékos mód: bizonyos játékokat akár egyedül is lehet játszani
- „Split-screen” mód: A felhasználó egy másik játékkal, a számítógép „átadásával” tud ugyan abban a játékban az első játékos ellen játszani
- Eredmények mentése és statisztikák megjelenítése

Archiváló funkciók:

- Az eredeti játékok tárolása modern, időtálló környezetben
- Kulturális érték és oktatási információ biztosítása a felhasználók számára

Rendszer összetevők:

- Webes felhasználói felület (frontend)
- Felhasználói fiókkezelés (regisztráció, bejelentkezés, profil személyre szabása)
- Adatbázis a játékok, eredmények és felhasználói adatok tárolásához
- Jövőbeli bővítések lehetősége moduláris felépítés révén

Technológiai elemek:

- Adatbázis-szerver: Felhasználói azonosítás és játékadatok kezelése
- Webes felület: Egyszerű, interaktív, felhasználóbarát UI a játékokhoz és archívumhoz
- Webes, illetve letölthető játékok
- Adminisztrációs elemek: a weboldal bizonyos tartalmainak moderálása
- Ranglista felület: regisztrált felhasználók legjobb eredményeinek publikus tárolása, melyet titkosítással lehet feltölteni adatokkal



2. Megrendelő bemutatása

A vállalkozás neve és profilja

Digitális Játékfejlesztő és Archiváló Kft. olyan innovatív vállalkozás, amely a digitális játékok fejlesztésére, modernizálására és archiválására specializálódott. Cégünk célja, hogy a klasszikus és modern játékelményt ötvözve egy olyan platformot hozzon létre, ahol a felhasználók nemcsak játszhatnak, hanem megőrizhetik a múlt digitális örökségét is. Tevékenységünk középpontjában a játékfejlesztés, a régi (retró) játékok felújítása és megőrzése, valamint a játékok hosszú távú digitális archiválása áll. A vállalkozás küldetése, hogy a játéktörténelem értékeit elérhetővé tegye a jövő generációi számára, miközben innovatív, webalapú játékelményt biztosít.

Tevékenységi kör

A Digitális Játékfejlesztő és Archiváló Kft. szolgáltatásai közé tartozik:

- **Digitális játékok fejlesztése és modernizálása:** Webes és mobil platformokra optimalizált, interaktív játékok fejlesztése, a klasszikus táblás és logikai játékok újragondolásával.
- **Játékarchiválás:** Régi, elfeledett vagy megszűnt online és offline játékok digitalizálása, dokumentálása és újradíszítése oktatási és kulturális célból.
- **Webalapú játékplatform üzemeltetése:** Saját fejlesztésű, felhőalapú rendszer, amelyen keresztül a felhasználók elérhetik az archívumban található játékokat és a modern fejlesztéseket egyaránt.
- **Testreszabott fejlesztési megoldások:** Egyedi szoftverfejlesztés és játékmechanikai tervezés oktatási intézmények, múzeumok vagy partnervállalatok számára.

A vállalkozás elsősorban a hazai piacon van jelen, de terjeszkedési tervei között szerepel a nemzetközi piacra való belépés is. A célközönség magában foglalja mind a fiatalabb, mind az idősebb korosztályt, akik számára a játékok nem csupán szórakozást, hanem értékes készségfejlesztést is képviselnek.

Szervezeti felépítés és méret

A Digitális Játékfejlesztő és Archiváló Kft. egy 15 fős dinamikus csapatból áll, amely a következő szerkezetben működik:



- Fejlesztői osztály (8 fő): Frontend és backend fejlesztők, akik felelősek a játékok technikai megvalósításáért.
- Tartalmi és design csapat (4 fő): Grafikusok és tartalomszervezők, akik a játékok felhasználói élményéért és vizuális világáért felelősek.
- Ügyfélszolgálat és marketing (3 fő): Ügyfélkapcsolatok kezelése, visszajelzések gyűjtése és a termékek promóciója.

A vállalkozás horizontális szervezeti felépítéssel rendelkezik, ami lehetővé teszi a gyors döntéshozatalt és a hatékony együttműködést.

Üzleti célok

A vállalkozás számára a fejlesztésre azért van szükség, hogy:

- Bővítse a játékportfólióját: Új, innovatív játékok bevezetésével vonzzon magasabb számú felhasználót
- Növelje a versenyképességet: A piacon egyedülálló, többféle játékmódot kínáló platform létrehozásával
- Digitalizálja a játékelményt: A hagyományos társasjátékokat modern, webes környezetbe helyezve, hozzáférhetőbbé téve azokat.
- Digitális kultúra megőrzését támogassa

Megrendelés indoklása

A Digitális Játékfejlesztő és Archiváló Kft. azért döntött szoftverünk mellett, mert az a következőket nyújtja:

- Sokoldalúság: A platform több különböző játékmódot kínál, ami széles körű célközönséget vonz.
- Adatbázis-kezelés: A felhasználói adatok biztonságos tárolása és kezelése, valamint a játékállapotok nyomon követése.
- Retró játék integráció: A retró játékok népszerűsége tovább növeli a platform vonzóerejét.

A választott szoftver fejlesztési megközelítés megfelel a vállalat technológiai irányelveinek, könnyen integrálható a meglévő infrastruktúrába, és hosszú távon skálázható.



Kapcsolat a projekttel

A fejlesztendő alkalmazás a Digitális Játékfejlesztő és Archiváló Kft céljait szolgálja: a játékelmény modernizálását, az oktatási lehetőségek támogatását, illetve a digitális játékörkörtség megőrzését.

A rendszer a cég portfóliójának egyik központi elemét képezi, több szinten is hozzájárulva a vállalati célok eléréséhez:

- **Felhasználói szinten:** könnyen hozzáférhető, biztonságos és élvezetes játékelményt biztosít egy- és többjátékos módban, ösztönzi a logikai, stratégiai készségek fejlesztését, és növeli a felhasználók aktivitását a platformon.
- **Adminisztrációs szinten:** egyszerűsíti a játékok és a digitális archívum kezelését, a játékok feltöltését, karbantartását és a felhasználói statisztikák nyomon követését.
- **Marketing szinten:** lehetőséget ad a felhasználói viselkedés és a játékelőzmények elemzésére, támogatva a célzott kampányokat, promóciókat és közösségi események szervezését.
- **Technológiai szinten:** integrálódik a meglévő webes környezetbe, adatbázis- és szerveroldali szolgáltatásokkal együttműködve, biztosítva a stabil, skálázható és biztonságos működést.

A projekt megvalósításával a Digitális Játékfejlesztő és Archiváló Kft. egy komplex, modern webalapú játékplatformot hozhat létre, amely ötvözi a játékelményt, az oktatási funkciókat és a digitális archívumot.

Az új rendszer nemcsak a felhasználók számára kínál kézzelfogható előnyöket, hanem a vállalat számára is stratégiai értéket teremt, hiszen bővíti a portfóliót, erősíti a felhasználói elköteleződést és elősegíti a cég hosszú távú fejlődését a digitális játékpiacon.



3. Követelmények és célkitűzések

Funkcionális követelmények

Játékok kiválasztása és indítása:

- Interaktív weboldal, mely reagál a felhasználó tevékenységeire (pl. egér mozgása)
- A felhasználó egy csempékből álló felületen választhatja ki a kívánt játékot, illetve a játék nevét a keresőmezőbe beírva is elérhetik a játék oldalait
- Egyes játékok letölthetők, míg a TRON webes környezetben fut

Játékmenet:

- Egyjátékos mód: a felhasználó gép ellen játszhat
- Többjátékos mód: játékok, melyek támogatják, „osztott képernyőn” való játékot
- Eredmények mentése és statisztikák generálása játékokban, melyek támogatják
- Eredmények feltöltése ranglistába

Felhasználói fiókok és interakciók:

- Webes felhasználói felület (frontend).
- Felhasználói fiókkezelés (regisztráció, bejelentkezés, személyre szabás)
- Adatbázis a játékok, eredmények és felhasználói adatok tárolásához
- Webes felület más játékosok eredményeinek megtekintéséhez
- Folyamatosan frissülő Ranglista felület
- Adminisztrációs elemek a weboldal moderálásához
- Jövőbeli bővítések lehetősége moduláris felépítés révén

További funkciók:

- Reszponzív (PC first) felület
- Személyre szabható felhasználói profilok melyeket adatbázis tárol
- Pontszám hozzáadása csakis titkosított formában



Nem funkcionális követelmények

Teljesítmény:

- A játékok betöltése legfeljebb 5-10 másodperc alatt

Biztonság:

- Felhasználói jelszavak titkosított tárolása
- GDPR és egyéb adatvédelmi előírások betartása

Skálázhatóság:

- Több száz egyidejű felhasználó kezelése
- Új játékok hozzáadása modulárisan, újrafeljesztés nélkül

Használhatóság és kompatibilitás:

- Egyszerű, intuitív kezelőfelület
- Népszerű böngészők támogatása
- Angol nyelv, később többnyelvű bővítés lehetősége

Karbantarthatóság:

- Kód moduláris, jól dokumentált
- Verziókezelés (pl. Git) használata a fejlesztés során



Felhasználói kör

Általános látogatók (vendég mód):

- Böngészhetik a játékokat
- Megtekinthetik a játék leírását és szabályait

Regisztrált felhasználók:

- Eredmények és statisztikák nyomon követése
- Saját profil testreszabása

Adminisztrátorok:

- Felhasználói eredmények kezelése
- Játékok és archívum moderálása
- Platform tartalmi karbantartása

Célcsoport

- Logikai és stratégiai játékok iránt érdeklődő felhasználók
- Diákok, fiatal felnőttek, valamint idősebbek, akik nosztalgikus vagy új játékelményt keresnek
- Oktatási intézmények, ahol a játékok segíthetik a tanulást és készségfejlesztést

A projekt határai

A rendszer tartalmazza:

- Reszponzív, felhasználóbarát felület
- 3 játék
- Ranglista
- Személyre szabható felhasználói fiók

A rendszer nem tartalmazza (későbbi célok):

- Külső fizikai eszközökkel való integrációt (pl. kontrollerek)
- Mobilalkalmazást (csak webes verzió, illetve .exe),
- Külső játék API-k integrációját
- Teljes körű közösségi funkciókat, online multiplayert (pl. chat)
- HTTPS kapcsolat használata (domain és certificate után)



4. Rendszerelemzés és tervezés

A jelenlegi helyzet elemzése

A digitális játékok világa hatalmas ütemben fejlődik, mégis kevés olyan weboldal létezik, amely átlátható, modern és közösségorientált, illetve ahol a felhasználók egyszerűen, biztonságosan és kényelmesen játszhatnak klasszikus vagy új játékokat. A jelenlegi online megoldások töredezték: egyes platformok csak egyetlen játékra épülnek, mások pedig nem biztosítanak letisztult felhasználói élményt vagy megbízható többjátékos környezetet.

A mai fiatal és digitálisan aktív korosztály már nem pusztán egy magányos, statikus játékelményt keres. Élményt, visszajelzést és interakciót várnak – gyors csatlakozási lehetőségekkel, átlátható felülettel, eredményekkel, kihívásokkal és akár mesterséges intelligencia által vezérelt ellenfelekkel. A jelenleg elérhető felületek ezt a fajta egységes, könnyen hozzáférhető játékelményt nem kínálják.

Egy modern webes játékplatform, amely egyszerre biztonságos, közösségi és könnyen bővíthető, képes választ adni a felhasználók sokféle igényére. A klasszikus játékok újraformálása, a stabil multiplayer lehetőségek és az egységes, intuitív felület lehetővé teszi, hogy mindenki könnyedén hozzáférjen és élvezhesse a digitális játékelményt, miközben aktívan részt vehet a közösségi interakciókban.

A megoldás koncepcionális alapjai

Platformunk újragondolja, hogy hogyan fedezik fel és élvezik az emberek a digitális játékokat, egy olyan központi helyet teremtve, ahol a klasszikus és eredeti alkotások egyaránt jelen vannak.

- **Dinamikus játékgyűjtemény:** A platform klasszikus játékok sorát kínálja, azonban lehetősége van a felhasználóknak az oldal által még nem támogatott játékok kérvényezésére.
- **Scoreboard:** Azon játékok, melyek támogatják, egy online ranglistával egészítik ki a játékelményt, melyen a felhasználók össze tudják hasonlítani saját eredményeiket másokéval.
- **Split-screen multiplayer:** Egyes játékok esetén lehetőség van egyszerre több játékosnak egy képernyőn játszani, növelve a közösségi élményt és a versengést.



- Letölthető játékok (offline): A klasszikus címek, például a sakk és a Space Shooter, teljesen letölthető .exe fájlként érhetők el, lehetővé téve a helyi játékot offline módban való használatát. Az eredménymentéshez internet kapcsolat szükséges.

Követelményelemzés

A rendszer követelményeinek meghatározásánál figyelembe kell vennünk, hogy bizonyos felhasználói csoportok milyen elvárásokkal fordulnak a platform felé.

Vendég felhasználók igényei

- Érdeklődnek a játékok iránt, megismernék a játékok működését szabályait.
- Gyors, rezponzív weboldalt és kellemes felhasználói élmény várnak el.

Regisztrált felhasználók igényei

- A regisztrált felhasználók elsősorban a játékok gyors töltését, kisimult, hibamentes működését igénylik.
- Vendégfelhasználókhöz hasonlóan igénylik a weboldal gyors, intuitív használatát
- Motivációjuk lehet a játékok élvezete, vagy „rekordok” megdöntése olyan játékokban, melyek támogatják a ranglista-rendszert.
- Fontos számukra a profiljuk személyre szabhatósága, mely a ranglistán mindenkinek elérhető, illetve ebből eredően a személyes adataik biztonságos tárolása külső felekkel szemben.

Szerepkörök (actors)

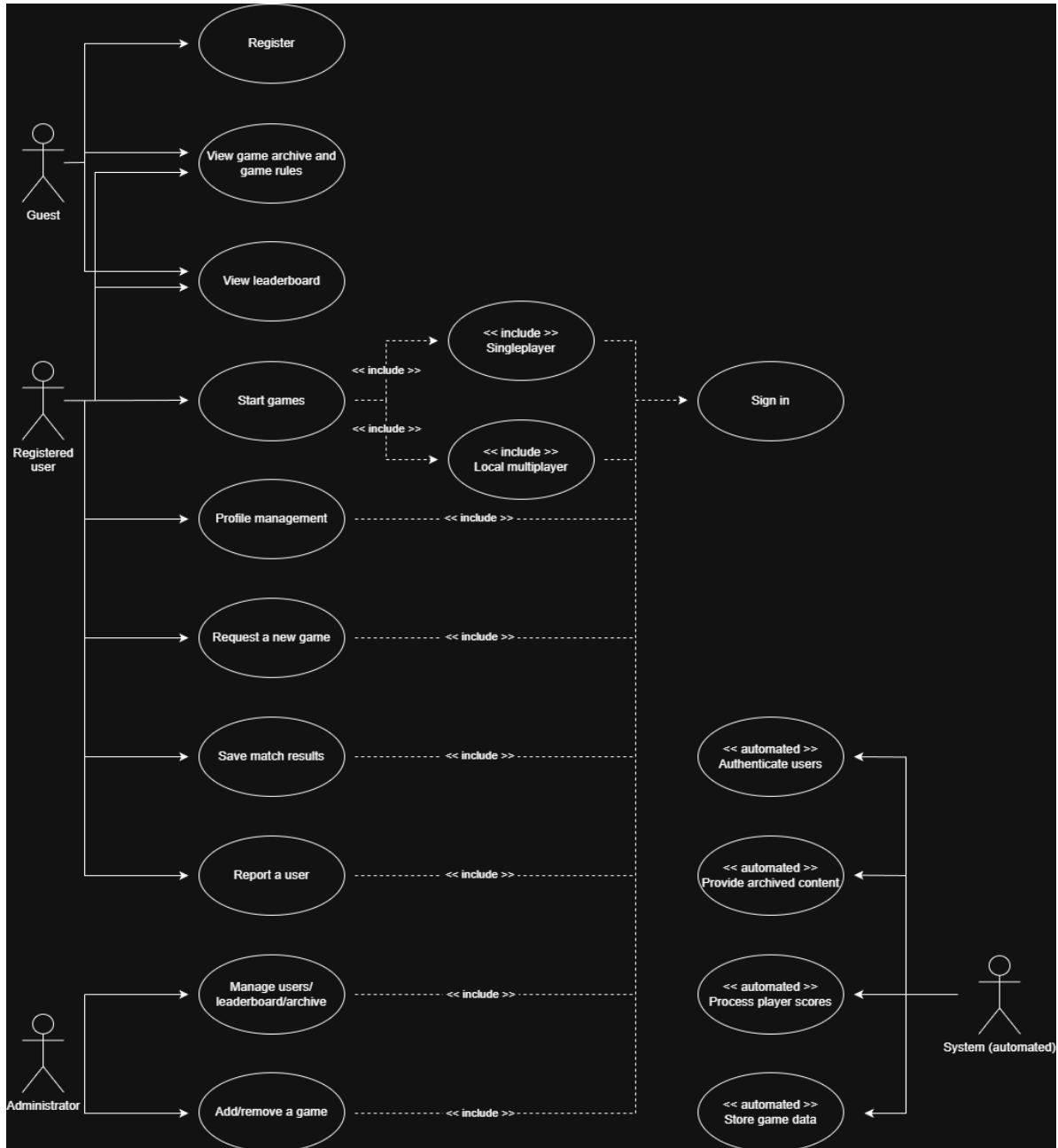
A rendszer két fő felhasználói szerepkört különböztet meg:

- **Vendég felhasználó:** Olyan felhasználó, aki nincs regisztrálva vagy bejelentkezve. Csak olvasási jogosultságokkal rendelkezik.
- **Regisztrált felhasználó:** Olyan felhasználók, akik saját e-mail címükkel regisztráltak, és az oldal többi funkcióját is igénybe tudják venni.
 - **Adminisztrátor:** Az oldal összes funkcióját használni tudja, beleértve a tartalom moderálását.

Használati esetek (use-cases)

A felhasználók és a rendszer főbb funkciói közötti kapcsolatok

use-case





Vendég felhasználó (guest)

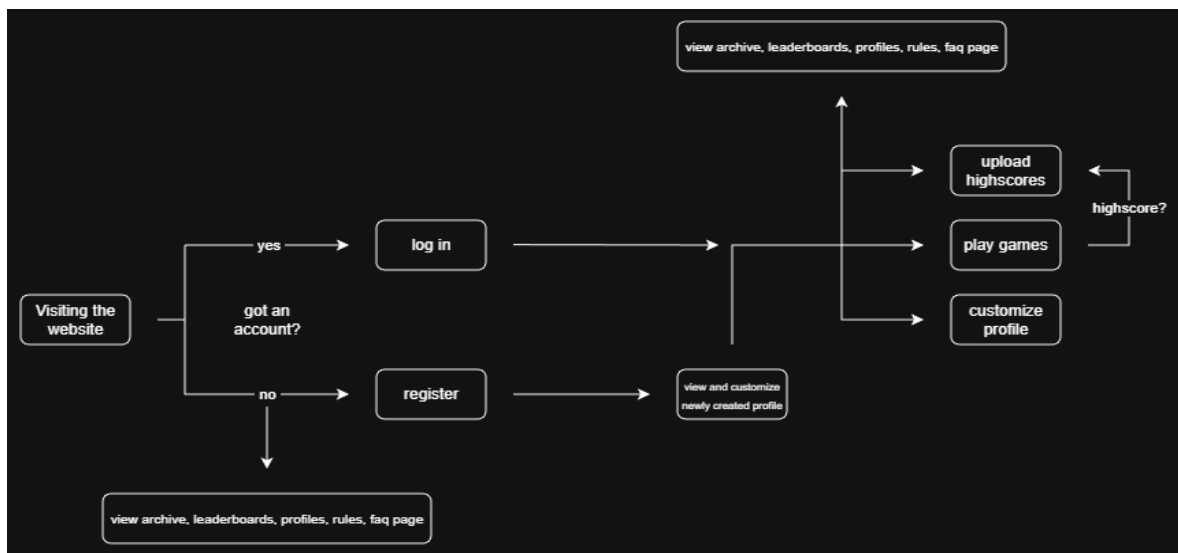
- Weboldal és archívum elérése
- Játékok és játékszabályok, illetve ranglisták böngészése
- Regisztráció

Regisztrált felhasználó (registered user)

- Weboldal és archívum elérése
- Játékok és játékszabályok, illetve ranglisták böngészése
- Játékok letöltése, elindítása
- Profil személyre szabása
- Játékok kérvényezése
- Eredmények mentése

Adminisztrátor

- Weboldal moderálása
- Játékok és játékszabályok, illetve ranglisták böngészése
- Játékok letöltése, elindítása
- Profil személyre szabása
- Eredmények moderálása
- Felhasználók moderálása





Projekt struktúrája

Az alkalmazás több technológiát használ: a frontend Angular keretrendszerre épül, a backend pedig PHP alapú, míg a játékok — a TRON kivételével (HTML, CSS, JavaScript, CANVAS) — C# nyelven készülnek. A projekt könyvtárszerkezete elkülöníti az egyes funkcionális egységeket, biztosítva az átláthatóságot és a későbbi bővíthetőséget.

A felhasználói felület külön komponensekből épül fel, például a navigációs sáv (`<app-navbar>`) és a lábléc (`<app-footer>`) önálló komponensként jelenik meg, amelyek az alkalmazás minden oldalán betöltődnek. A játékok megjelenítése egy közös nézeten történik, ahol a rendszer a játék egyedi azonosítója (ID) alapján tölti be a megfelelő tartalmat.

A routing az Angular Router segítségével van megvalósítva. A játékkoldal egy dinamikus útvonalon keresztül érhető el, amely a játék azonosítója alapján jeleníti meg a megfelelő játékot vagy letöltési lehetőséget.

Frontend (Angular):

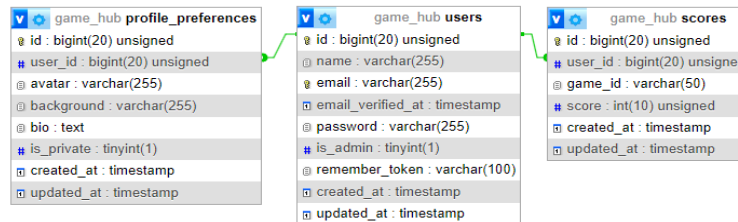
- A teljes felhasználói felület Angular komponensekből áll.
- Komponensek, szolgáltatások (Services) és modulok külön mappákban találhatók.
- A fő modulok kezelik a navigációt, a játékok indítását és a felhasználói interakciókat.

A játékok egységes adatszerkezettel kerülnek kezelésre, amely minden játék esetében tartalmazza az alapvető adatokat (azonosító, név, típus, leírás, elérési mód). Ez lehetővé teszi, hogy a webes és letölthető játékok kezelése azonos logika mentén történjen.

A játékokhoz hasonlóan a felhasználói adatokat és statisztikákat is külön modellek határozzák meg.

Backend (PHP):

- A backend a MyPHP környezetre épül, REST API-kat biztosít a frontend számára.
- A vezérlők (Controllers) kezelik a HTTP kéréseket és válaszokat, míg a modellek (Models) végzik az adatbázis-műveleteket.
- Az üzleti logika főként a vezérlőkben és a modellekben valósul meg, például a felhasználói profiladatok és pontszámok kezelésénél.



Játék modulok (C#, web)

- A *Sakk* és a *Space Shooter* játékok C# nyelven készültek és önálló alkalmazások. Ezek a játékok a webes platformhoz kapcsolódnak, amely biztosítja a hozzáférést, a leírásokat és – a Space Shooter esetén – az eredmények feltöltését és megjelenítését a ranglistán.
- A TRON játék teljes egészében webes technológiákkal (HTML, CSS, JavaScript, CANVAS) készült, és közvetlenül a böngészőben fut. A játék egy HTML canvas alapú arénában valósul meg, ahol két játékos egymás ellen tud játszani ugyan azon a billentyűzetet használva.
- A webes platform egységes felületen kezeli mind a böngészőben futó játékokat, mind a letölthető alkalmazásokat, biztosítva a felhasználói fiókok, statisztikák és ranglisták központi kezelését.

Repository és Policy osztályok

- A jogosultságkezelés a backend oldalon történik, például az admin funkciók és a felhasználók saját adatainak módosítása esetén. A rendszer biztosítja, hogy csak a megfelelő jogosultsággal rendelkező felhasználók hajthassanak végre bizonyos műveleteket (pl. pontszám törlése, profil módosítása).

Ez a struktúra biztosítja a moduláris, karbantartható és skálázható projekt felépítést, miközben lehetővé teszi a különböző technológiák – Angular, PHP, C# – zökkenőmentes együttműködését.



5. Feladatok meghatározása és erőforrás-tervezés

Erőforrás-tervezés

Emberi erőforrások - Fejlesztői csapat:

- Árvai Márk: Frontend fejlesztés, UI/UX design, TRON játéklogika, backend integráció, napi 1-5 óra
- Galó Gellért: Backend fejlesztés, adatbázis tervezés, sakk és space shooter logika, heti 2-3 óra

Technikai erőforrások - Fejlesztői környezet:

- Visual Studio Code + Git
- PhpStorm
- C#
- MySQL adatbázis

Pénzügyi erőforrások

Költségvetés: 0 Ft (nyílt forráskódú technológiák)

- Fejlesztői eszközök: ingyenes
- Hosting: ingyenes tier (GitHub)
- Adatbázis: ingyenes MySQL
- Domain: nem szükséges

Feladatok meghatározása

Frontend fejlesztés, backend integráció, dokumentáció (Árvai Márk)

A weboldal esztétikájának megtervezése (glassmorphism)

Főoldal és navigációs rendszer kialakítása:

- Navbar
- Footer
- Tiles

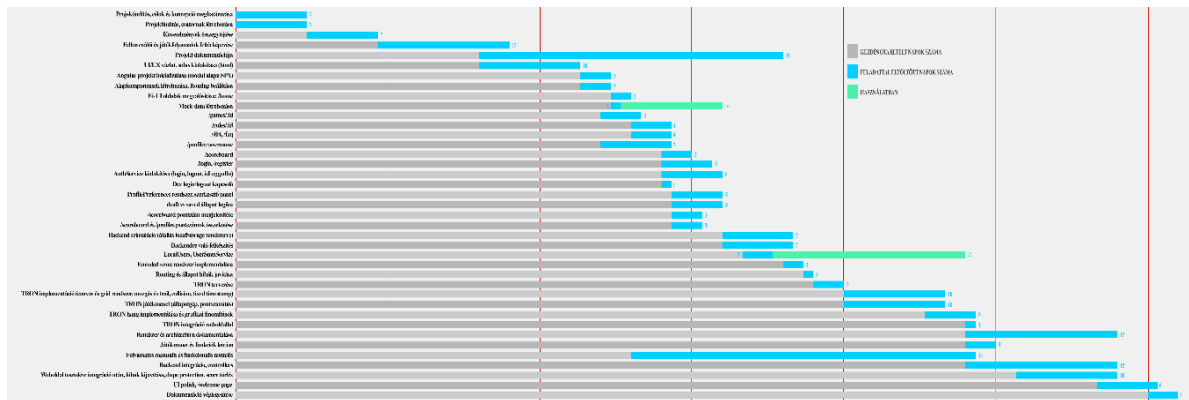
Felhasználói felület komponensek tervezése:

- Home page

- Leaderboard page
- Profile page
- Games page
- Rules page
- FAQ page
- Request page
- Welcome/Landing page
- Reszponzív (PC first) design implementálása
- API kommunikáció kezelése
- API integráció tesztelése

TRON játék modul

- Játéklogika implementálása („motorok” elhelyezése, collision detection, pálya renderelése)
- UI komponensek aréna, játékosok és „fénycsíkok” megjelenítése, pontszámok)
- Játékállapot kezelése
- Játék vége és eredmények megjelenítése



Márk's Gantt

<https://imgur.com/a/HUz07v1> (teljes kép)

Backend fejlesztés (Galó Gellért)

Laravel

- REST API végpontok tervezése és implementálása
- Adatbázis séma kialakítása és migrációk



Sakk játék modul:

- Sakkszabályok motorjának fejlesztése
- Játékállapot kezelése és validálása

Space Shooter játék modul:

- Játékfizika implementálása
- Ütközésetektálás
- Score rendszer

Időterv (2025.11.01 – 2026.01.26)

1. Előkészítő és tervezési fázis (2025.11.01 – 2025.11.07)

Projekt céljának és terjedelmének meghatározása

Játékarchívum koncepció kidolgozása

Technológiai stack kiválasztása (Angular, Laravel, MySQL)

Fejlesztői környezet beállítása

Alap architektúra és mappastruktúra megtervezése

2. Alaprendszer kialakítása (2025.11.08 – 2025.11.30.)

- Angular projekt inicializálása
- Globális layout kialakítása (navbar, router-outlet, footer)
- Routing rendszer megvalósítása
- Home oldal és játékkártyák kialakítása
- Mock adatszerkezetek létrehozása (játékok, felhasználók, pontszámok)

3. Funkcionális modulok fejlesztése (2025.12.01 – 2026.01.05.)

- Games oldal dinamikus működésének megvalósítása
- Rules oldal kialakítása
- Scoreboard oldal és rangsorolási logika implementálása



- Profil oldal megvalósítása
- Profil testreszabás (avatar, háttér, privát mód)
- AuthService és LocalUsers rendszer kialakítása
- Bejelentkezés és regisztráció oldalak elkészítése

4. TRON játék fejlesztése (2025.12.01 – 2026.01.05)

- Játéktér és rácsmodell kialakítása
- Játéklogika (mozgás, ütközésetekeltálás)
- Játékállapot-kezelés (kör, meccs, vége)
- Pontozási rendszer
- Hanghatások integrálása
- Felhasználói felület és overlayek kialakítása

5. Backend integráció és finomhangolás (2026.01.01 – 2026.01.15)

- Backend API végpontok illesztése a frontendhez
- Felhasználói adatok és pontszámok backend kezelése
- Duplikációvédelem implementálása
- Jogosultságkezelés (admin pontszám törlés)
- Frontend és backend közötti adatcsere tesztelése

6. Tesztelés és dokumentáció (2026.01.10 – 2026.01.26)

- Funkcionális tesztelés (UI, profil, játék, scoreboard)
- Hibák javítása
- Felhasználói dokumentáció elkészítése
- Fejlesztői dokumentáció elkészítése
- Végző rendszerellenőrzés



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



Kockázatkezelés

- Iteratív fejlesztés rövid szakaszokban
- Rendszeres mentések és verziókezelés
- Funkciók fokozatos integrálása
- Problémás modulok újratervezése szükség esetén

Teljesítménymutatók

- Git commit aktivitás
- Elkészült funkcionális modulok száma
- Hibák számának csökkenése
- Dokumentáció elkészültsége



6. Fejlesztői környezet és eszközök

Fejlesztői környezet (IDE-k, keretrendszerek, programozási nyelvek)

A projekt fejlesztése több különböző technológia és fejlesztői környezet együttes használatával történt. Frontend oldalon a rendszer Angular keretrendszerrel készült, TypeScript, HTML és CSS nyelvek használatával.

A fejlesztéshez az alábbi környezetek és eszközök kerültek alkalmazásra:

- Angular CLI 20.3.9
- Node.js 22.17.1
- npm 11.5.1
- Visual Studio Code (frontend fejlesztéshez)
- Adobe Photoshop (grafikai elemek és háttérképek készítéséhez)

Backend oldalon a rendszer Laravel keretrendszerre épül, PHP nyelven implementálva. Az adatkezelés MySQL adatbázis segítségével történik.

A backend fejlesztéshez az alábbi eszközök kerültek használatra:

- PHP (Laravel framework)
- Laravel Sanctum (hitelesítéshez)
- PhpStorm (backend fejlesztéshez)
- XAMPP (lokális fejlesztőkörnyezet biztosítására)
- MySQL adatbázis-kezelő rendszer

A játékmódulok különböző technológiákkal készültek:

- A TRON játék webes technológiákkal (HTML, CSS, JavaScript, Canvas)
- A Sakk és Space Shooter játékok C# nyelven készültek önálló alkalmazásként

Verziókezelés (Git, GitHub használata, commit struktúra)

A projekt verziókezelése Git segítségével történt, a forráskód tárolása és megosztása GitHub felületen valósult meg.

A verziókezelés célja:

- a módosítások nyomon követése,



- a fejlesztési folyamat biztonságossá tétele,
- valamint a hibák visszakövethetőségének biztosítása.

A commitok során törekvés volt az egyértelmű és funkcióorientált megnevezések használata, például:

- új funkciók hozzáadása,
- hibajavítások elkülönítése,
- UI módosítások külön commitban történő rögzítése.

A GitHub lehetővé tette a közös munkát, valamint a projekt folyamatos mentését és archiválását.

Adatbázis-kezelő rendszer

Az alkalmazás adatainak tárolására MySQL adatbázis került felhasználásra.

Az adatbázis főbb funkciói:

- felhasználói adatok tárolása (név, e-mail, jelszó),
- profilbeállítások (avatar, háttérkép, bemutatkozó szöveg, privát profil beállítás),
- pontszámok és ranglista adatok kezelése.

A backend Laravel keretrendszer Eloquent ORM rendszert használ az adatbázissal való kommunikációhoz. Az adatbázis szerkezete relációs modellre épül, ahol a felhasználók, profiladatok és pontszámok külön táblákban kerülnek tárolásra, kapcsolatokkal összekötve.

Tesztelési eszközök (unit tesztek, rendszertesztek, hibakövető eszközök)

A frontend tesztelés manuális formában történt.

Frontend oldalon a következő tesztelési lépések valósultak meg:

- regisztráció és bejelentkezés tesztelése,
- helytelen e-mail és jelszóformátumok ellenőrzése,
- játékok indításának és letöltésének tesztelése,
- pontszám feltöltésének ellenőrzése,
- profil testreszabásának (avatar, háttérkép, privát mód) tesztelése,
- kijelentkezés és újrabejelentkezés ellenőrzése.



A TRON játék tesztelése során:

- két játékos irányításának ellenőrzése,
- ütközésetektlés tesztelése,
- pontszámítás ellenőrzése,
- játékállapotok (indítás, kör vége, játék vége) tesztelése történt.

Backend oldalon az API végpontok működésének ellenőrzése valósult meg, különös tekintettel:

- hitelesítésre,
- pontszám beküldésére,
- duplikált pontszámok kezelésére.

Automatikus tesztelőeszköz (pl. Postman) használata nem történt, a működés ellenőrzése valós használati esetekkel történt.

Segédprogramok (grafikai, dokumentációs vagy UI-tervező eszközök)

A projekt fejlesztése során több segédprogram is alkalmazásra került:

Grafikai eszközök:

- Adobe Photoshop – háttérképek és grafikai elemek készítéséhez, egyedi, nagy felbontású (3440×3440) háttérképek létrehozása oldalanként.

Dokumentációs eszközök:

- Microsoft Word – fejlesztői és felhasználói dokumentáció elkészítéséhez,
- képernyőképek készítése a működés bemutatására.
- GitHub
- Microsoft Teams

Egyéb segédeszközök:

- böngésző fejlesztői eszközök (Chrome DevTools) hibakereséshez,
- helyi szerver környezet (XAMPP) a backend futtatásához és teszteléséhez.



7. Megvalósítás

A projekt megvalósításának áttekintése

A projekt fejlesztése 2025. november elején indult. Az eredeti cél egy olyan webalapú játékkarchívum létrehozása volt, amely egységes felületen biztosít hozzáférést több különböző játékhoz, valamint lehetőséget ad a felhasználói profilok kezelésére és az eredmények nyilvántartására.

A fejlesztés során a rendszer két fő részre bontható:

- frontend alkalmazás (Angular alapú webes felület),
- backend szolgáltatás (Laravel alapú REST API).

A megvalósítás során elsődleges cél volt egy olyan architektúra kialakítása, amely:

- moduláris felépítésű,
- később bővíthető,
- backend-kompatibilis,
- jól elkülöníti az egyes funkcionális egységeket.

Frontend és TRON játék megvalósítása (Árvai Márk)

Kezdeti tervezés

A fejlesztés első lépéseként a felhasználói felület megtervezése történt meg egyszerű HTML és CSS segítségével. Ennek célja az oldal szerkezetének és vizuális stílusának meghatározása volt.

Eredetileg a projekt egy Torpedó játékot is tartalmazott volna, azonban a fejlesztés során kiderült, hogy egyetlen képernyőn nehezen játszható több játékos számára. Ezért a játékmódul lecserélésre került egy TRON típusú játékra, amely helyi kétjátékos módban, billentyűzetről jól irányítható.

Ezután a statikus HTML alapú terv Angular keretrendszerbe került átültetésre.



Angular alkalmazás felépítése

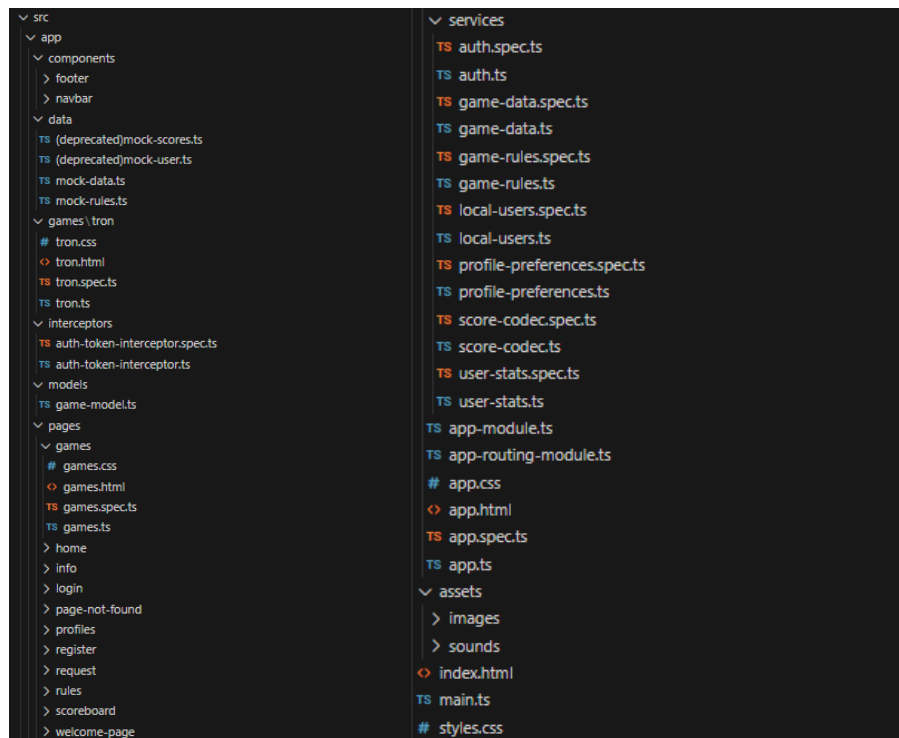
Az alkalmazás Angular keret rendszerrel, modul-alapú struktúrában készült el. A fő szerkezeti elemek:

- külön komponensek a navigációs sávhoz (navbar),
- külön komponens a lábléchez (footer),
- külön oldalkomponensek (home, games, profiles, scoreboard, rules, info, request, login, register, welcome).
- külön komponens TRON-nak

Az alkalmazás globális felépítése:

```
<app-navbar>  
<router-outlet>  
<app-footer>
```

Könyvtárszerkezete:

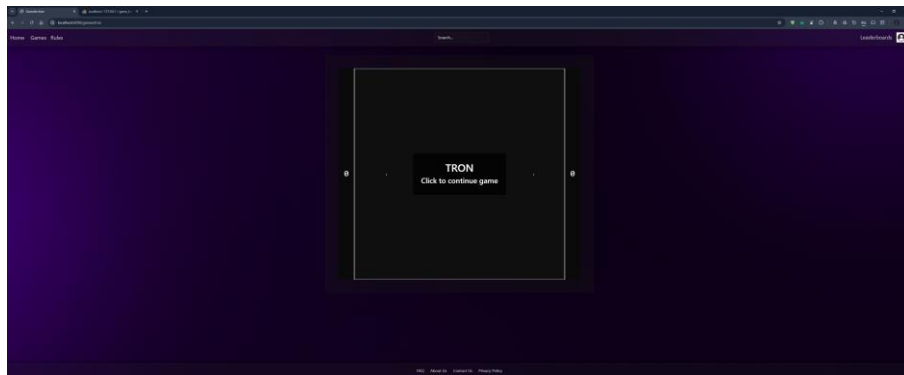




A navigáció Angular Router segítségével valósul meg. A játéköldalak, szabályoldalak, illetve felhasználói profiloldalak dinamikus útvonalakon keresztül érhetők el:

- `/games/:id`
- `/rules/:id`
- `/profiles/:username`

Ez lehetővé teszi, hogy egyetlen komponens több különböző játékot kezeljen az azonosító alapján.



`/games/tron`

Routing:

```
const routes: Routes = [  
  { path: "login", component: Login },  
  { path: "register", component: Register },  
  { path: "home", component: Home },  
  { path: "request", component: Request },  
  { path: "scoreboard", component: Scoreboard },  
  { path: "profiles/:username", component: Profiles },  
  { path: "games/:id", component: Games },  
  { path: "404", component: PageNotFound },  
  { path: "rules/:id", component: Rules },  
  { path: "info", component: Info },  
  { path: "welcome", component: WelcomePage },  
  { path: "", redirectTo: "/welcome", pathMatch: "full" },  
  { path: "**", redirectTo: "404", pathMatch: "full" }  
];
```

Felhasználói hitelesítés (Auth rendszer)

A fejlesztés kezdeti szakaszában a felhasználói adatok és a bejelentkezési állapot kezelése localStorage segítségével történt, amely a backend működését szimulálta. Ez lehetővé tette a frontend funkcióinak (regisztráció, bejelentkezés, profilkezelés) önálló tesztelését valódi szerver nélkül.



A későbbi fejlesztési fázisban ez a megoldás lecserélésre került a Laravel alapú REST API-ra, így a rendszer fokozatosan átállt valódi kliens–szerver kommunikációra.

A frontend oldalon külön Auth szolgáltatás került megvalósításra, amely:

- kezeli a bejelentkezett felhasználó állapotát,
- eltárolja a bejelentkezési adatokat,
- biztosítja a visszairányítást (return URL) bejelentkezés után.

A rendszer:

- regisztrációt
- bejelentkezést
- kijelentkezést támogat

A jogosultságkezelés nem route guardokkal, hanem felhasználói felületen történő megjelenítési logikával van megoldva (*ngIf).

```
<ng-container *ngIf="game?.id === 'tron' && auth.isLoggedIn; else normalPage">  
  <div class="game-stage">  
    <app-tron></app-tron>  
  </div>  
</ng-container>
```

Az alábbi kódrészlet a frontend oldali hitelesítési szolgáltatás egy részét mutatja be. A kód kezeli a bejelentkezést és kijelentkezést, valamint nyilvántartja a bejelentkezett felhasználó nevét és admin jogosultságát. A szolgáltatás HTTP kéréseken keresztül kommunikál a backend REST API végpontjaival.

```
login(email: string, password: string) {  
  return this.http.post<any>(`${Auth.API}/login`, { email, password });  
}  
  
logout(): void {  
  this.http.post(`${Auth.API}/logout`, {}).subscribe(() => {  
    localStorage.removeItem('auth.token');  
    this.usernameSubject.next(null);  
    this.isAdminSubject.next(false);  
    this.router.navigate(['/login']);  
  });  
}  
  
get isLoggedIn(): boolean {  
  return !!this.usernameSubject.value;  
}
```

```
private fetchMe(): void {  
  this.http.get<any>(`${Auth.API}/me`).subscribe(res => {  
    this.usernameSubject.next(res.username);  
    this.isAdminSubject.next(!res.isAdmin);  
  });  
}
```



Profilrendszer megvalósítása

A profiloldalak a /profiles/:username útvonalon érhetők el.

A rendszer különbséget tesz:

- saját profil
- más felhasználó profilja között

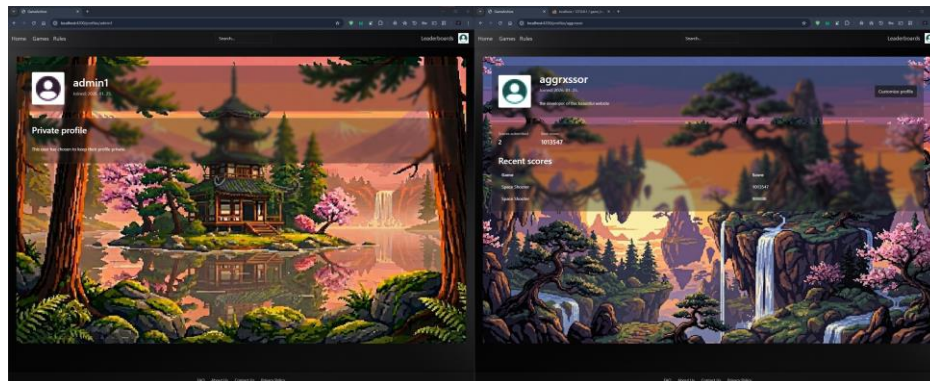
A profil funkciói:

- avatar kiválasztása
- háttérkép kiválasztása
- bemutatkozó szöveg
- privát profil beállítása
- statisztikák megjelenítése

A profil testreszabás külön szolgáltatáson keresztül történik (ProfilePreferences Service).

A mentett adatok:

- avatar
- háttérkép
- privát státusz
- bio



private vs public profile, különböző profile preferences (pl bio, háttér, avatar)



Ranglista (Scoreboard)

A ranglista oldalon jelennek meg a játékosok által elért pontszámok.

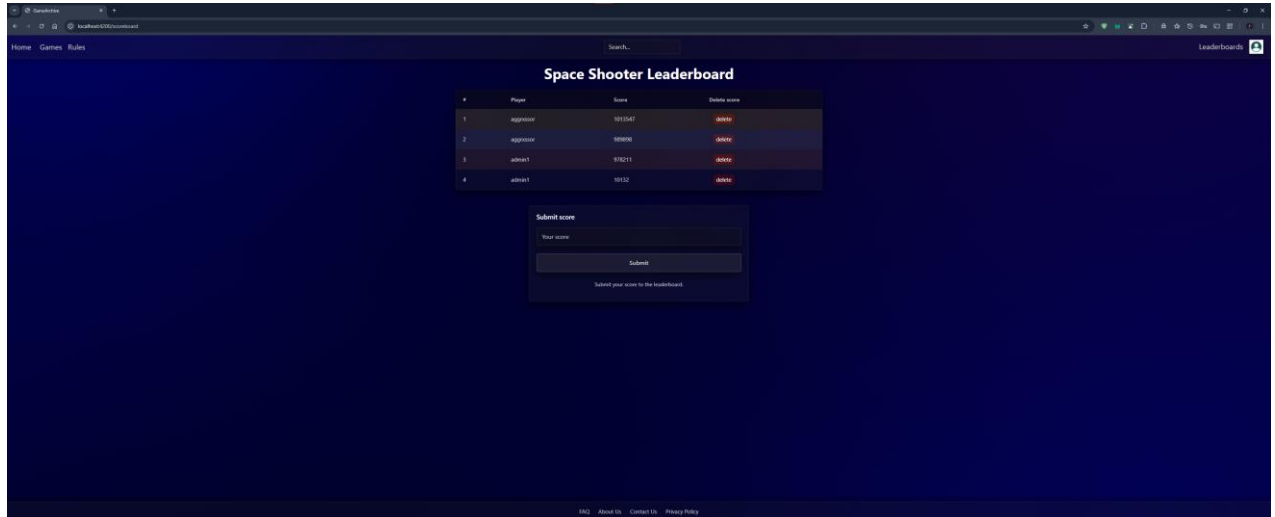
Funkciói:

- pontszám feltöltése
- pontszámok rendezése csökkenő sorrendben
- profilra mutató linkek megjelenítése
- admin jogosultság esetén törlés lehetősége

A pontszámok feltöltése kódolt formában történik, amely előkészíti a rendszert későbbi csalás elleni védelemre.

A rendszer:

- frontend oldalon ellenőrzi a bemenetet
- backend oldalon is végez duplikáció ellenőrzést



TRON játék megvalósítása

A TRON játék teljes egészében webes technológiákkal készült:

- HTML canvas
- JavaScript
- Angular komponensbe ágyazva



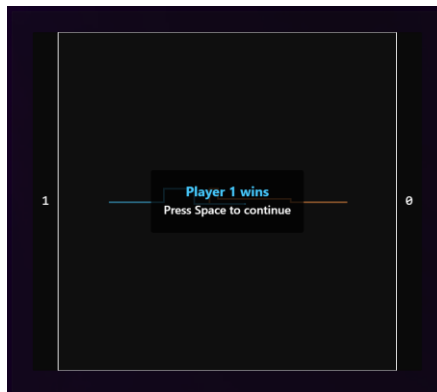
Főbb elemei:

- rácsalapú pálya
- két játékos (WASD és nyilak)
- ütközésetektálás
- pontszámítás
- több körös mérkőzés (3 nyert kör)

A játék állapotgéppel működik:

- betöltés
- várakozás
- visszaszámlálás
- játék
- kör vége
- játék vége

A grafika teljes egészében Canvas rajzolással történik. A játék állapottól függően képes hangok lejátszására is (pl. deathSound.wav)



Felhasználói felület (UI)

A teljes weboldal megjelenése egységes vizuális stílust követ:

- glassmorphism (üveg hatás),
- áttetsző elemek,
- zajmintás textúra,
- egyedi háttér minden oldalhoz.



A háttérképek egyedileg, Adobe Photoshop segítségével készültek, 3440×3440 felbontásban.

A rendszer részben reszponzív:

- kisebb képernyőkön hamburger menü jelenik meg,
- az elemek függőlegesen rendeződnek.

Frontend és backend integráció

A frontend Angular HttpClient segítségével kommunikál a backenddel.

Az adatcsere JSON formátumban történik.

A szolgáltatások (services):

- elkülönítik a megjelenítést az adatkezeléstől,
- megkönnyítik a későbbi backend cserét vagy bővítést.

Tesztelés

A tesztelés manuális módon történt.

Frontend tesztek:

- regisztráció
- bejelentkezés
- profil testreszabás
- pontszám feltöltés
- kijelentkezés és újrabejelentkezés
- hibás email/jelszó
- nem létező fiók

TRON játék tesztelése:

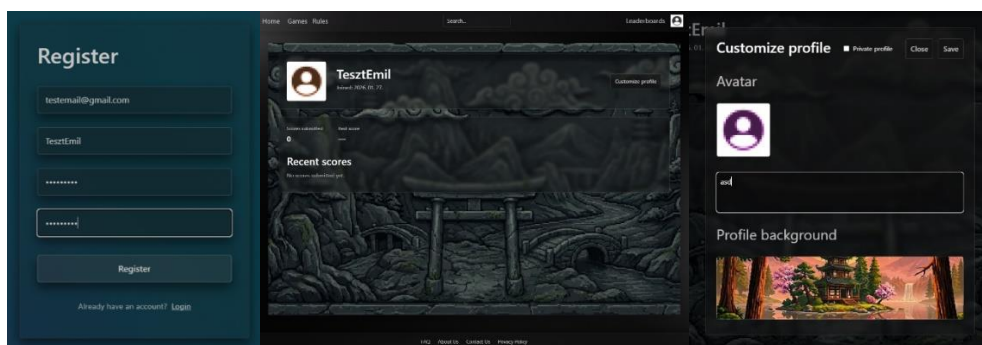
- irányítás ellenőrzése
- ütközések vizsgálata
- körváltás
- pontszámítás
- „döntetlen” tesztelés

Backend tesztelés:

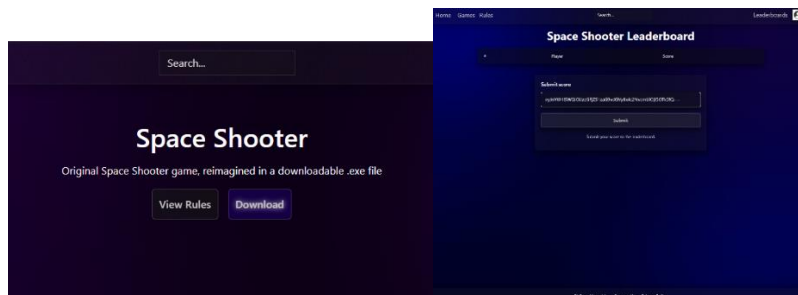
- API végpontok működésének ellenőrzése
- jogosultságkezelés ellenőrzése
- duplikált pontszámok kezelése
- score törlése/visszavonása

Példa frontend tesztelésre

Regisztrációt követően a weboldal automatikusan a frissen regisztrált felhasználó profiljára vezeti a felhasználót, ahol azt személyre lehet szabni.



Mentés után az elérhető játékok egyikét kiválasztja, majd eredményét feltölti Ranglistára.



Az eredmény megjelenik a ranglistán.





A regisztrált felhasználó, illetve az eredménye megjelenik az adatbázisban.

Edit

Copy

Delete

3 TesztEmil testemail@gmail.com NULL \$2y\$12\$WwAD1W8X6IT2bc9P9eKusEYqZfCeXF5EZID04toE6... 0 NULL

↩

→

id

user_id

game_id

score

created_at

updated_at

Edit

Copy

Delete

13 space-shooter299992026-01-27 18:58:152026-01-27 18:58:15

Backend fejlesztés (Galó Gellért)

A backend fejlesztés során egy teljes körű, REST API-alapú szerveroldali rendszer készült, amely a webes játékkönyvtár háttérlogikáját, adatkezelését és integrációs pontjait biztosítja. A fejlesztés Laravel PHP keretrendszerrel történt, MySQL adatbázis használatával. A forráskód a GitHubon érhető el:

A backend főbb feladatai a következők voltak:

1. REST API tervezés és implementáció

A backend több végpontot biztosít, amelyeken keresztül a frontend kommunikálhat a szerverrel. Ezek a végpontok az alábbi kategóriákba sorolhatók.

Felhasználókezelés (Authentication & User Management)

- POST /api/register – Új felhasználó regisztrálása. Az e-mail és jelszó ellenőrzése után a rendszer létrehozza a felhasználót és visszaad egy hozzáférési tokenet.
- POST /api/login – Bejelentkezés. Ellenőrzi a hitelesítési adatokat, és sikeres bejelentkezés esetén Sanctum tokenet ad vissza.
- POST /api/logout – Kijelentkezés. A felhasználó aktuális tokenjének érvénytelenítése.
- GET /api/user – A bejelentkezett felhasználó adatainak lekérdezése (név, e-mail, profilbeállítások).
- PUT /api/user/profile – A felhasználói profil frissítése (avatar URL, háttérkép, bemutatkozás, privát profil beállítása).

Játékadatok kezelése (Game Management)

- GET /api/games – Az összes elérhető játék listázása alapadatokkal (név, leírás, típus, elérési útvonal).



- GET /api/games/{id} – Egy adott játék részletes adatainak lekérése.

Pontszámok és ranglista (Scoreboard & Leaderboard)

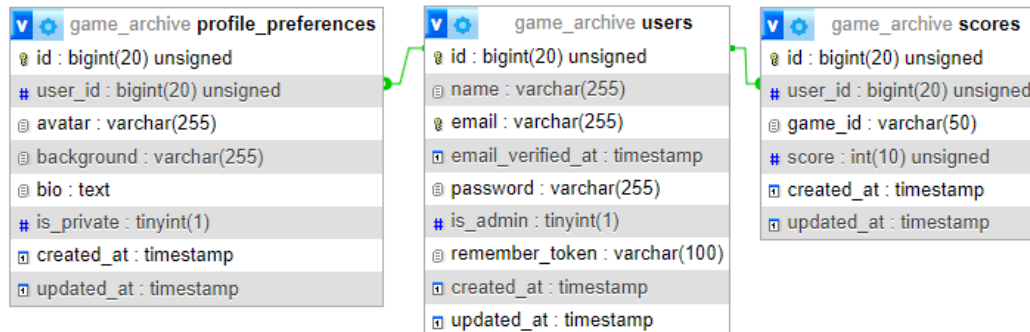
- POST /api/scores – a frontend kódolt (nem nyers) formában küldi a pontszámot, amelyet a backend dekódol és ellenőriz, ellenőriz és eltárol.
- GET /api/scores – A ranglista lekérése, opcionális szűréssel játék vagy felhasználó szerint.
- DELETE /api/scores/{id} – Pontszám törlése admin jogosultság esetén.

Játékkérések kezelése (Game Requests)

- POST /api/requests – Felhasználók által beküldött játékkérések fogadása.
- GET /api/requests – A beküldött kérések listázása admin jogosultsággal.

2. Adatbázis tervezés és migrációk

Az adatbázis relációs modellre épül, és az alábbi fő táblákat tartalmazza:



A migrációk Laravel Migration fájlok segítségével készültek, amely biztosítja az adatbázis-séma verziókezelhetőségét és egyszerű telepíthetőségét.

3. Hitelesítés és jogosultságkezelés (Laravel Sanctum)

A backend oldalon a felhasználói hitelesítés Laravel Sanctum segítségével került megvalósításra, amely tokenalapú autentikációt biztosít.



A jogosultságkezelés middleware-ekkel történik, például az admin funkciók és a felhasználók saját adatainak módosításának korlátozására.

A frontend jelenleg részben integrált módon használja a backend hitelesítési mechanizmusait, a hozzáférésvezérlés elsősorban felhasználói felületi (UI) szinten történik

4. Játékmódulok integrációja (Chess és Space Shooter)

A Sakk és Space Shooter játékok különálló C# alkalmazásként készültek, amelyek a webes platformról (GitHub) érhetők el. A backend fő feladata ezek esetében a pontszámkezelése volt.

Space Shooter játék

A játék végén a pontszám kimásolható kódolt formátumban. A backend ellenőrzi annak érvényességét, kiszűri a duplikációkat, majd eltárolja az adatbázisban. A pontszám titkosított formában érkezik, ami csökkenti a csalás lehetőségét.

6. Biztonsági intézkedések és GDPR megfelelés

A rendszer biztonsága kiemelt szerepet kapott. A jelszavak bcrypt algoritmussal kerülnek titkosításra. Az API tokenek kezelése Sanctum segítségével történik.

A rendszer architektúrája fel van készítve a felhasználói adatok törlésére és exportálására, amely a GDPR követelményeinek megfelelően bővíthető.

7. Tesztelés és hibakezelés

A backend tesztelése manuálisan történt, az alábbi területeken:

- API végpontok működésének ellenőrzése,
- hitelesítés és jogosultságkezelés vizsgálata,
- hibás adatok és duplikált pontszámok kezelése,
- adatbázis migrációk ellenőrzése.



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



Összegzés

A projekt során sikerült egy teljes funkcionalitású, több játékot kezelő webes alkalmazást megvalósítani.

A kitűzött célok nagy része megvalósult:

- játékarchívum,
- felhasználói rendszer,
- ranglista,
- egy játszható böngészős játék.

Nem került megvalósításra:

- teljes admin felület,
- játékfeltöltő rendszer.

A rendszer architektúrája azonban alkalmas ezek későbbi bevezetésére.



8. Felhasználói dokumentáció

A rendszer elindítása

A rendszer használatához a backend és a frontend egyaránt szükséges.

Szükséges előfeltételek

A futtatáshoz az alábbi szoftverek telepítése szükséges:

- Node.js és NPM
- PHP és Composer
- MySQL adatbázis
- XAMPP (Apache + MySQL)

Backend indítása

1. Indítsuk el az XAMPP alkalmazást, majd kapcsoljuk be:

- Apache
- MySQL szolgáltatást

2. A projekt backend mappájában futtassuk:

```
composer install
```

3. Adatbázis migráció futtatása:

```
php artisan migrate
```

4. Amennyiben a rendszer rákérdez az adatbázis létrehozására:

Would you like to create it? → yes

5. A backend szerver indítása:

```
php artisan serve
```

A backend ezután a következő címen fut: `http://localhost:8000`



Frontend indítása

- 1. Új terminálablak megnyitása**
- 2. Navigálás a game-archive mappába**
- 3. Függőségek telepítése:**

```
npm install
```

- 4. Frontend indítása:**

```
ng serve
```

A weboldal ezután az alábbi címen érhető el: <http://localhost:4200>

A telepítésről képes útmutató elérhető GitHub-on (readme.md).

A weboldal használata

1. Regisztráció

- A főoldalon a „Register” menüpont segítségével új felhasználó hozható létre.
- A regisztráció során meg kell adni:
 - e-mail címet
 - felhasználónevet
 - jelszót

Sikeres regisztráció után a rendszer automatikusan a felhasználó profiloldalára irányít.

2. Bejelentkezés

A regisztrált felhasználók a „Login” oldalon tudnak belépni az adataik megadásával.

3. Profil testreszabása

A profiloldalon lehetőség van:

- profilkép (avatar) kiválasztására
- háttérkép beállítására
- bemutatkozó szöveg megadására
- a profil privát módba állítására

A módosítások mentése után a rendszer eltárolja az adatokat az adatbázisban.



4. Játékok böngészése

A „Games” menüpontban:

- elérhető az összes játék listája
- keresőmező segíti a játékok gyors megtalálását
- a játékok külön oldalon jelennek meg leírással és indítási lehetőséggel

A játék típusa lehet:

- böngészőben futó játék (pl. TRON)
- letölthető játék (pl. Space Shooter)

5. Játék indítása és eredmények mentése

Egy játék elindítása után:

- a felhasználó végigjátssza a játékot
- a játék végén az eredményt feltöltheti a ranglistára (submit score)

Az eredmény:

- a felhasználó nevéhez kötődik
- megjelenik a megfelelő játék ranglistáján

6. Ranglista megtekintése

A „Leaderboard” oldalon:

- a felhasználók pontszámai jelennek meg
- csökkenő sorrendben vannak rendezve
- a játékos neve kattintható, amely a profiljára vezet

Admin jogosultság esetén:

- lehetőség van pontszám törlésére



Budapesti Műszaki Szakképzési Centrum
Újpesti Két Tanítási Nyelvű Műszaki Technikum
1041 Budapest IV. ker., Görgey Artúr utca 26.
OM azonosító: 203058/013



7. Kijelentkezés

A „Logout” gomb segítségével a felhasználó kijelentkezhets a rendszerből.

Hibakezelés

- Hibás e-mail vagy jelszó esetén a rendszer figyelmeztető üzenetet jelenít meg.
- Ha egy nem létező játék oldalára navigál a felhasználó, a rendszer 404-es oldalt jelenít meg.
- Hálózati hiba esetén a rendszer nem engedi az adatküldést, és erről tájékoztatja a felhasználót.



8. Fejlesztői dokumentáció

A rendszer architektúrája

A Game Archive projekt egy kliens–szerver architektúrára épülő webalkalmazás. A frontend Angular keretrendszerrel készült, amely egyoldalas alkalmazásként (SPA) működik, míg a backend Laravel alapú REST API-t biztosít. Az adatok MySQL adatbázisban kerülnek tárolásra.

A frontend és a backend között a kommunikáció HTTP protokollon keresztül, JSON formátumban történik. A hitelesítés token alapú, Laravel Sanctum segítségével, amely lehetővé teszi, hogy a frontend minden védett végpontot azonosított felhasználóként érjen el.

Ez a felépítés biztosítja, hogy:

- a frontend és backend külön fejleszthető
- a rendszer könnyen bővíthető új funkciókkal
- az üzleti logika nem kerül kliensoldalra
- a felhasználói adatok védettek maradnak

Routing rendszer

A navigáció az Angular Router segítségével történik. Az útvonalak az app-routing-module.ts fájlban kerülnek definiálásra.

```
const routes: Routes = [  
  { path: "login", component: Login },  
  { path: "register", component: Register },  
  { path: "home", component: Home },  
  { path: "request", component: Request },  
  { path: "scoreboard", component: Scoreboard },  
  { path: "profiles/:username", component: Profiles },  
  { path: "games/:id", component: Games },  
  { path: "404", component: PageNotFound },  
  { path: "rules/:id", component: Rules },  
  { path: "info", component: Info },  
  { path: "welcome", component: WelcomePage },  
  { path: "", redirectTo: "/welcome", pathMatch: "full" },  
  { path: "**", redirectTo: "404", pathMatch: "full" }  
];
```



A rendszer támogatja a dinamikus útvonalakat, például:

- games/:id
- profiles/:username
- rules/:id

Ez lehetővé teszi, hogy ugyanaz a komponens több különböző tartalmat jelenítsen meg az URL paramétereinek alapján. A routing konfiguráció tartalmaz továbbá:

- alapértelmezett átirányítást
- 404-es (page not found) oldalt

Ez a megoldás biztosítja a folyamatos felhasználói élményt oldalújrátöltés nélkül.

Hitelesítés és tokenkezelés

A frontend hitelesítési logikája az AuthService osztályban valósul meg. Ez a szolgáltatás kezeli:

- a bejelentkezést
- a regisztrációt
- a kijelentkezést
- a felhasználó állapotát

A felhasználói állapot BehaviorSubject segítségével kerül tárolásra, így az alkalmazás bármely része feliratkozhat az aktuális felhasználónévre és jogosultságra.

```
private usernameSubject = new BehaviorSubject<string | null>(null);  
private isAdminSubject = new BehaviorSubject<boolean>(false);
```

A fetchMe() metódus gondoskodik arról, hogy az oldal frissítése után is megmaradjon a bejelentkezett állapot, amennyiben a token megtalálható a localStorage-ben.

```
private fetchMe(): void {  
  this.http.get<any>(`${Auth.API}/me`).subscribe(res => {  
    this.usernameSubject.next(res.username);  
    this.isAdminSubject.next(!res.isAdmin);  
  });  
}
```


A register() metódus a backend /register végpontját hívja meg, majd a kapott token-t elmenti, és automatikusan a felhasználó profiloldalára navigál.

```
register(username: string, email: string, password: string): void {  
  this.http.post<any>(`${Auth.API}/register`, { name: username, email, password })  
    .subscribe(res => {  
      this.handleAuth(res);  
      this.router.navigate(['/profiles', res.user.username]);  
    });  
}
```

A logout() metódus törli a token-t, visszaállítja az állapotot és átirányítja a felhasználót a login oldalra.

```
logout(): void {  
  this.http.post(`${Auth.API}/logout`, {}).subscribe(() => {  
    localStorage.removeItem('auth.token');  
    this.usernameSubject.next(null);  
    this.isAdminSubject.next(false);  
    this.router.navigate(['/login']);  
  });  
}
```

Login és regisztráció oldalak működése

A login oldal logikája a login.ts fájlban található. A metódus:

- ellenőrzi az üres mezőket
- meghívja az AuthService login metódusát
- kezeli a sikeres és hibás válaszokat
- támogatja a returnUrl logikát

```
login(): void {  
  this.validationError = null;  
  this.authError = null;  
  
  if (!this.email.trim() || !this.password.trim()) {  
    this.triggerValidationError('Email and password required');  
    return;  
  }  
  
  this.auth.login(this.email, this.password).subscribe({  
    next: res => {  
      this.auth.handleAuth(res);  
  
      const returnUrl =  
        this.route.snapshot.queryParamMap.get('returnUrl') || '/home';  
  
      this.router.navigateByUrl(returnUrl);  
    },  
    error: () => {  
      this.triggerAuthError('Incorrect email or password');  
    }  
  });  
}
```

A regisztrációs oldal kliensoldali validációt végez:

- e-mail formátum ellenőrzés
- felhasználónév hosszának ellenőrzése
- jelszó hossz és egyezőség vizsgálata

```
register(): void {
  this.validationError = null;

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const usernameRegex = /^[a-zA-Z0-9_-]{6,}$/;

  if (!emailRegex.test(this.email)) {
    this.triggerError('Incorrect email');
    return;
  }

  if (!usernameRegex.test(this.username)) {
    this.triggerError(
      'Usernames must be at least 6 characters long and can only use letters, numbers, - and _'
    );
    return;
  }

  if (this.password.length < 8) {
    this.triggerError('Password must be at least 8 characters long');
    return;
  }

  if (this.password !== this.confirmPassword) {
    this.triggerError('Passwords do not match');
    return;
  }

  this.auth.register(this.username, this.email, this.password);
}

private triggerError(message: string): void {
  this.validationError = message;
  this.flashError = true;

  setTimeout(() => {
    this.flashError = false;
  }, 500);
}
```

Játékoldalak és dinamikus betöltés

A játékoldalak a games.ts komponensben kerülnek feldolgozásra. Az aktuális játék az URL paraméteréből (:id) kerül kiválasztásra.

```
ngOnInit(): void {
  this.route.paramMap.subscribe(params => {
    const id = params.get('id');
    if (!id) {
      this.game = undefined;
      return;
    }

    this.gameData.getById(id).subscribe(game => {
      this.game = game;
    });
  });
}
```

Letölthető játék esetén a rendszer ellenőrzi, hogy a felhasználó be van-e jelentkezve, és szükség esetén átirányítja a login oldalra.

```
onDownloadClick(event: Event): void {  
  if (!this.auth.isLoggedIn) {  
    event.preventDefault();  
    this.router.navigate(['/login'], {  
      queryParams: {  
        returnUrl: this.router.url  
      }  
    });  
  }  
}
```

Profiloldal működése

A profiloldal a rendszer egyik legösszetettebb része. A betöltési folyamat több lépésből áll:

1. Felhasználó adatainak lekérése a backendből
2. Profilbeállítások betöltése
3. Alapértelmezett értékek használata hiba esetén

```
ngOnInit(): void {  
  this.route.paramMap.subscribe(params => {  
    const username = params.get('username');  
    if (!username) return;  
  
    this.username = username;  
  
    this.auth.getProfile(username).subscribe({  
      next: apiUser => {  
        this.prefs.get(apiUser.username).subscribe({  
          next: prefs => {  
            const defaults = this.prefs.defaults();  
  
            this.isPrivateProfile = prefs.isPrivate ?? false;  
  
            this.user = {  
              username: apiUser.username,  
              joined: apiUser.joined,  
              avatar: prefs.avatar ?? defaults.avatar,  
              background: prefs.background ?? defaults.background,  
              bio: prefs.bio ?? ''  
            };  
  
            this.currentAvatar = this.user.avatar;  
            this.currentBackground = this.user.background;  
            this.selectedAvatar = this.currentAvatar;  
            this.selectedBackground = this.currentBackground;  
            this.bio = prefs.bio ?? '';  
  
            this.computeOwnership();  
            this.recomputeStats();  
          },  
          error: () => {  
            const defaults = this.prefs.defaults();  
  
            this.user = {  
              username: apiUser.username,  
              joined: apiUser.joined,  
              avatar: defaults.avatar,  
              background: defaults.background,  
              bio: ''  
            };  
  
            this.currentAvatar = defaults.avatar;  
            this.currentBackground = defaults.background;  
            this.selectedAvatar = defaults.avatar;  
            this.selectedBackground = defaults.background;  
            this.bio = '';  
  
            this.computeOwnership();  
            this.recomputeStats();  
          }  
        });  
      },  
      error: () => {  
        this.user = undefined;  
      }  
    });  
  });  
}
```

A rendszer kiszámítja, hogy a megnyitott profil a bejelentkezett felhasználóé-e, és csak ebben az esetben engedi a szerkesztést.

```
private computeOwnership(): void {  
    this.isOwnProfile = false;  
  
    if (this.user && this.auth.isLoggedIn && this.auth.username) {  
        this.isOwnProfile =  
            this.auth.username.toLowerCase() === this.user.username.toLowerCase();  
    }  
}
```

A profil mentésekor a kiválasztott értékek kerülnek elküldésre a backend felé.

```
saveProfileCustomization(): void {  
    if (!this.user || !this.isOwnProfile) return;  
  
    this.currentAvatar = this.selectedAvatar;  
    this.currentBackground = this.selectedBackground;  
  
    this.prefs.save({  
        avatar: this.currentAvatar,  
        background: this.currentBackground,  
        bio: this.bio,  
        isPrivate: this.isPrivateProfile,  
    }).subscribe();  
}
```

Ranglista és pontfeltöltés

A ranglista betöltése a scoreboard.ts komponensben történik. A backend által visszaadott adatokat a frontend átalakítja megjeleníthető struktúrára.

```
loadLeaderboard(): void {  
    this.stats.getLeaderboard('space-shooter').subscribe(items => {  
        this.scores = items.map(i => ({  
            username: i.username,  
            score: i.score,  
            scoreId: i.scoreId  
        }));  
    });  
}
```

A pontfeltöltés egy kódolt formátumon keresztül történik. A ScoreDecoder dekódolja és ellenőrzi a bemenetet, majd továbbítja a backendnek.

```
decode(encoded: string): DecodedScore | null {
  try {
    const json = atob(encoded);
    const data = JSON.parse(json);

    if (typeof data.gameId !== 'string') return null;
    if (typeof data.score !== 'number') return null;
    if (!Number.isFinite(data.score) || data.score < 0) return null;

    return {
      gameId: data.gameId,
      score: data.score,
    };
  } catch {
    return null;
  }
}
```

```
submitEncodedScore(input: HTMLInputElement): void {
  if (!this.auth.isLoggedIn) {
    this.router.navigate(['/login'], {
      queryParams: {
        returnUrl: this.router.url
      }
    });
    return;
  }

  this.submitError = null;

  const raw = input.value.trim();
  if (!raw) return;

  const decoded = this.decoder.decode(raw);
  if (!decoded) {
    this.submitError = 'Invalid score code.';
    return;
  }

  this.stats.submitScore(decoded.gameId, decoded.score).subscribe({
    next: () => {
      input.value = '';
      this.loadLeaderboard();
    },
    error: err => {
      if (err.status === 422 && err.error?.message) {
        this.submitError = err.error.message;
      } else {
        this.submitError = 'Failed to submit score.';
      }
    }
  });
}
```

Admin jogosultság esetén lehetőség van pontszám törlésére, valamint visszavonására (undo).

```
confirmDelete(): void {  
  if (!this.pendingDelete) return;  
  
  this.stats.deleteScore(this.pendingDelete.scoreId).subscribe(() => {  
    this.lastDeleted = {  
      username: this.pendingDelete!.username,  
      score: this.pendingDelete!.score,  
      scoreId: this.pendingDelete!.scoreId,  
      gameId: 'space-shooter',  
      date: new Date().toISOString()  
    };  
  });  
  
  this.pendingDelete = null;  
  this.loadLeaderboard();  
  
  if (this.undoTimeoutId) {  
    clearTimeout(this.undoTimeoutId);  
  }  
  
  this.undoTimeoutId = setTimeout(() => {  
    this.lastDeleted = null;  
  }, 10000);  
});  
}  
  
cancelDelete(): void {  
  this.pendingDelete = null;  
}  
  
undoDelete(): void {  
  if (!this.lastDeleted) return;  
  
  this.stats.submitScore(this.lastDeleted.gameId, this.lastDeleted.score)  
    .subscribe(() => {  
      this.lastDeleted = null;  
      this.loadLeaderboard();  
    });  
}
```

TRON játék technikai megvalósítása

A TRON játék egy külön Angular komponensben (tron.ts) került megvalósításra. A játék HTML canvas alapú megjelenítést használ.



A játék fő elemei:

- rácsalapú pálya
- két játékos
- mozgásirányok
- ütközésvizsgálat
- pontszámítás

A játékciklus időalapú, requestAnimationFrame és akkumulátoros léptetés segítségével működik.

```
/* ----- rendereles ----- */

ngAfterViewInit() {
  this.setVolume();

  this.canvas.nativeElement.focus();
  this.resizeCanvas();
  this.ctx = this.canvas.nativeElement.getContext('2d')!;
  this.ctx.imageSmoothingEnabled = false;

  window.addEventListener('keydown', this.keyHandler);

  this.rafId = requestAnimationFrame(this.loop);
}

loop = (t: number) => {
  if (!this.lastTime) this.lastTime = t;
  const delta = (t - this.lastTime) / 1000;
  this.lastTime = t;
  this.accumulator += delta;

  while (this.accumulator >= this.MOVE_INTERVAL) {
    if (this.phase === 'running' && !this.matchOver) this.step();
    this.accumulator -= this.MOVE_INTERVAL;
  }

  this.render();
  this.rafId = requestAnimationFrame(this.loop);
}
```

Az ütközésvizsgálat figyel:

- fálnak ütközést
- másik játékos nyomvonalát
- frontális ütközést

```
// lépések fixed
step() {
  const p1 = this.player1;
  const p2 = this.player2;

  if (!p1.alive && !p2.alive) return;

  // irány változtatás
  if (p1.nextDir !== p1.dir) {
    const diff = Math.abs(p1.nextDir - p1.dir);
    if (diff !== 2) p1.dir = p1.nextDir;
  }

  if (p2.nextDir !== p2.dir) {
    const diff = Math.abs(p2.nextDir - p2.dir);
    if (diff !== 2) p2.dir = p2.nextDir;
  }

  const nx1 = p1.x + this.dx[p1.dir];
  const ny1 = p1.y + this.dy[p1.dir];

  const nx2 = p2.x + this.dx[p2.dir];
  const ny2 = p2.y + this.dy[p2.dir];

  let p1Dead = false;
  let p2Dead = false;

  // fallal
  if (nx1 < 0 || nx1 >= this.GRID_W || ny1 < 0 || ny1 >= this.GRID_H) p1Dead = true;
  if (nx2 < 0 || nx2 >= this.GRID_W || ny2 < 0 || ny2 >= this.GRID_H) p2Dead = true;

  // vonalal
  if (!p1Dead && this.grid[this.idx(nx1, ny1)] !== 0) p1Dead = true;
  if (!p2Dead && this.grid[this.idx(nx2, ny2)] !== 0) p2Dead = true;

  // frontális utkozes eseten
  if (nx1 === nx2 && ny1 === ny2) {
    p1Dead = true;
    p2Dead = true;
  }

  if (p1.alive) this.grid[this.idx(p1.x, p1.y)] = 1;
  if (p2.alive) this.grid[this.idx(p2.x, p2.y)] = 2;

  p1.alive = !p1Dead;
  p2.alive = !p2Dead;

  if (p1Dead || p2Dead) {
    this.endGame();
    return;
  }

  p1.x = nx1;
  p1.y = ny1;
  p2.x = nx2;
  p2.y = ny2;
}
```

A körök végén a rendszer frissíti a pontszámot, és ellenőrzi, hogy a mérkőzés véget ért-e.


```
endGame() {  
  if (this.phase === 'roundOver' || this.phase === 'gameover') return;  
  
  this.play(this.soundDeath);  
  this.phase = 'roundOver';  
  
  if (!this.player1.alive && !this.player2.alive) {  
    this.winner = 0;  
  }  
  else if (this.player1.alive && !this.player2.alive) {  
    this.winner = 1;  
    this.score1++;  
  }  
  else if (this.player2.alive && !this.player1.alive) {  
    this.winner = 2;  
    this.score2++;  
  }  
  else {  
    this.winner = 0;  
  }  
  
  if (this.score1 === 3 || this.score2 === 3) {  
    this.matchOver = true;  
    this.phase = 'gameover';  
    this.play(this.soundWin);  
  }  
  
  this.player1.started = false;  
  this.player2.started = false;  
}
```

Szolgáltatások (Services)

A szolgáltatások biztosítják a backendhez való hozzáférést. Ezek:

- UserStats
- ProfilePreferences
- GameData
- GameRules

```
getLeaderboard(gameId: string): Observable<GameScore[]> {  
  return this.http.get<GameScore[]>(  
    `${UserStatsService.API}/leaderboard/${gameId}`  
  );  
}  
  
getStats(username: string) {  
  return this.http.get<{  
    gamesPlayed: number;  
    bestScore: number | null;  
    recentScores: { gameId: string; score: number }[];  
}>(`http://127.0.0.1:8000/api/stats/${username}`);  
}  
  
submitScore(gameId: string, score: number) {  
  return this.http.post(  
    'http://127.0.0.1:8000/api/scores',  
    { gameId, score }  
  );  
}  
  
deleteScore(scoreId: string): Observable<any> {  
  return this.http.delete(  
    `${UserStatsService.API}/scores/${scoreId}`  
  );  
}  
  
get(username: string): Observable<ProfilePrefs> {  
  return this.http.get<any>(`${this.apiUrl}/${username}`).pipe(  
    map(data => ({  
      avatar: data.avatar,  
      background: data.background,  
      bio: data.bio,  
      isPrivate: data.is_private  
    })))  
}
```

Biztonság és hibakezelés

A frontend:

- nem tárol jelszót
- token használ hitelesítéshez
- ellenőrzi a bemeneteket
- kezeli az API hibákat

```
intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {  
  const token = localStorage.getItem('auth.token');  
  
  if (!token) {  
    return next.handle(req);  
  }  
  
  const authReq = req.clone({  
    headers: {  
      Authorization: `Bearer ${token}`  
    }  
  });  
  
  return next.handle(authReq);  
}
```

A hibás kérések felhasználóbarát üzenetekkel jelennek meg.



Összegzés a fejlesztői megvalósításról

A rendszer moduláris, jól strukturált Angular frontenddel rendelkezik, amely REST API-n keresztül kommunikál egy Laravel backenddel. A kód elkülöníti:

- a megjelenítést
- az üzleti logikát
- az adatkezelést

Ez a felépítés lehetővé teszi a projekt további bővítését, például új játékok hozzáadását, új hitelesítési módok bevezetését vagy valós idejű funkciók implementálását.

9. Továbbfejlesztési lehetőségek

A rendszer jelenlegi állapota stabil alapot biztosít további funkciók fejlesztéséhez. Az egyik legfontosabb bővítési irány az online többjátékos mód megvalósítása. Ez WebSocket vagy Socket.io alapú kommunikációval lenne kivitelezhető, amely lehetővé tenné a valós idejű játékos-játékos interakciót, például közös játékmeneteket, élő ranglistafrissítést vagy többjátékos játékmódokat (például online TRON vagy sakk). A meglévő REST API erre jó alapot biztosít, mivel a felhasználói azonosítás és a pontszámkezelés már megvalósított.

További fejlesztési lehetőség egy teljes körű adminisztrációs felület kialakítása. Jelenleg az admin jogosultság főként pontszámok törlésére szolgál, azonban egy külön admin panel segítségével kezelhetővé válhatnának a felhasználók, a játéklista, a beküldött játékkérések, valamint a ranglisták. Ez lehetővé tenné a rendszer tartalmának moderálását és karbantartását felhasználóbarát módon.

A projekt bővíthető lenne mobilra optimalizált játékokkal, illetve egy külön mobilalkalmazás készítésével is. A meglévő backend újrahasznosítható lenne Android vagy iOS kliensekhez, így a rendszer nemcsak webes, hanem mobil platformon is elérhetővé válna. A reszponzív felület továbbfejlesztésével a játékelmény kisebb kijelzőkön is kényelmesebbé válhatna.

Egy további bővítési irány a „tools” jellegű funkciók bevezetése, például egy olyan eszköz, amely 2D rajzokból voxel-alapú 3D modelleket generál. Ez lehetőséget adna kreatív tartalmak készítésére, amelyeket a felhasználók megoszthatnának vagy akár játékbeli objektumként is felhasználhatnának. Egy ilyen funkció a rendszer játékgyűjtemény jellegét kiegészítené egy alkotóeszköz szerepkörrel. Ennek a fejlesztése már elkezdődött, ez azonban nem került bele a projektbe.



Végül a rendszer folyamatosan bővíthető új játékok hozzáadásával, különböző műfajokban (logikai, ügyességi, stratégiai játékok). A jelenlegi architektúra lehetővé teszi, hogy új játékok viszonylag egyszerűen integrálhatók legyenek, mivel a játékadatok egységes szerkezetben kerülnek kezelésre. Ez hosszú távon biztosíthatja a projekt fejlődését és a felhasználói bázis növekedését.

10. Összegzés

A Game Archive projekt célja egy olyan webes alkalmazás létrehozása volt, amely egységes felületen teszi lehetővé játékok megjelenítését, a felhasználók kezelését és az eredmények nyomon követését. A projekt során egy Angular alapú frontend és egy Laravel alapú backend került kialakításra, amelyek REST API-n keresztül kommunikálnak egymással.

A fejlesztés során sikerült megvalósítani a legfontosabb funkciókat: a regisztrációt és bejelentkezést, a profilkezelést, a játékok dinamikus megjelenítését, valamint a pontszámok feltöltését és ranglistán történő megjelenítését. A rendszer moduláris felépítése lehetővé teszi az egyes funkciók elkülönített fejlesztését és a későbbi bővítést.

A projekt nemcsak működő alkalmazást eredményezett, hanem fejlesztői szempontból is hasznos tapasztalatot adott az Angular és a Laravel használatáról, az API-alapú kommunikációról, valamint a jogosultságkezelés és biztonság alapelveiről. A megvalósított architektúra jó alapot biztosít további funkciók hozzáadásához és egy összetettebb, nagyobb terhelést is kiszolgáló rendszer kialakításához.

Összességében a projekt elérte a kitűzött céljait, és egy olyan stabil alapot hozott létre, amely alkalmas további fejlesztésekre és egy valós környezetben történő alkalmazásra is.



11. Források

- Angular dokumentáció
<https://angular.io/docs>
- Angular Router dokumentáció
<https://angular.io/guide/router>
- Laravel keretrendszer dokumentáció
<https://laravel.com/docs>
- Laravel Sanctum dokumentáció
<https://laravel.com/docs/sanctum>
- PHP dokumentáció
<https://www.php.net/docs.php>
- MySQL dokumentáció
<https://dev.mysql.com/doc/>
- REST API tervezési alapelvek
<https://restfulapi.net/>
- HTML5 Canvas API dokumentáció
https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- TypeScript dokumentáció
<https://www.typescriptlang.org/docs/>
- GitHub – Game Archive projekt forráskód
<https://github.com/aggrxssor/game-archive>
- SVG Noise Generator
<https://www.fffuel.co/nnnoise/>
<https://noice.vercel.app/>
<https://www.w3schools.com/>
<https://stackoverflow.com/questions>